

Project 5: Django Task Tracker CI/CD Demo

Overview

This project demonstrates deploying a Python Django web application using Jenkins. The key learning objective is ****Multi-stage Docker builds for web frameworks****. Multi-stage builds help keep the final production image lightweight by discarding build-time dependencies.

Prerequisites

Ensure the following are installed and running on your Jenkins server:

- Docker
- Trivy
- Python 3 & pip

Step 1: Network Setup

Create a shared Docker network for your infrastructure and deployments:

```
docker network create jenkins-net
```

Step 2: Infrastructure Setup

Run SonarQube and Jenkins on the `jenkins-net` network:

```
# Run SonarQube
```

```
docker run -d --name sonarqube -p 9000:9000 --network jenkins-net  
sonarqube:lts-community
```

```
# Run Jenkins (with Docker socket mounted)
```

```
docker run -d --name jenkins -p 8080:8080 -p 50000:50000 --network  
jenkins-net -v /var/run/docker.sock:/var/run/docker.sock -v  
jenkins_home:/var/jenkins_home jenkins/jenkins:lts
```

Step 3: Jenkins Configuration

1. Log into Jenkins.
2. Install necessary plugins: Docker Pipeline, SonarQube Scanner, Pipeline.
3. Configure SonarQube Server:
 - Go to Manage Jenkins -> System.
 - Add a SonarQube server named SonarQube and point it to `http://sonarqube:9000`.
4. Configure Global Tools:
 - Go to Manage Jenkins -> Tools.
 - Add a SonarQube Scanner installation.

Step 4: Pipeline Execution

1. Create a new Pipeline job in Jenkins.
2. Choose "Pipeline script from SCM" or paste the contents of Jenkinsfile.
3. Run the build. Take a close look at the "Docker Multi-Stage Build" stage in the console output to see how the image is built in two distinct phases!

Step 5: Verification

Access the deployed Django application:

- URL: <http://localhost:8000/tasks/>
- Expected Output: `{"tasks": ["Setup Jenkins", "Write Dockerfile", "Deploy Django App"]}`